

LAPORAN PRAKTIKUM PEKAN 7

STRUKTUR DATA

Disusun Oleh:

Muhammad Fharel

2511531010

Dosen Pengampu:

Dr. Wahyudi S.T.M.T

Asisten Pratikum:

Muhammad Galid Avero



DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

2026

KATA PENGANTAR

Segala puji penulis panjatkan ke hadirat Allah SWT atas rahmat dan karunia-Nya sehingga laporan praktikum Struktur Data pada tanggal 19 Mei 2026 yang membahas tentang beberapa metode pengurutan data seperti Insertion Sort, Selection Sort, dan Bubble Sort. Selain itu, penulis juga mempelajari penerapan GUI (Graphical User Interface) untuk memvisualisasikan proses sorting langkah demi langkah sehingga lebih mudah dipahami.

Ucapan terima kasih ditujukan kepada dosen pengampu, asisten praktikum, serta rekan-rekan yang telah membantu dalam proses pelaksanaan praktikum. Penulis menyadari bahwa penulisan laporan masih jauh dari kata sempurna, oleh karena itu kritik dan saran sangat diharapkan untuk penyempurnaan di kemudian hari. Semoga laporan ini memberikan manfaat dan menambah wawasan pembaca.

Padang, 20 Mei 2026

Penulis

DAFTAR ISI

KATA PENGANTAR.....	i
DAFTAR ISI.....	ii
BAB I PENDAHULUAN.....	1
1.1 Latar Belakang.....	1
1.2 Tujuan Praktikum	1
1.3 Manfaat Praktikum	2
BAB II PEMBAHASAN	3
2.1 Sorting Algorithm.....	3
2.2 Kode Pemograman	4
2.2.1 Class Insertion Sort	4
2.2.2 Output Insertion Sort.....	5
2.2.3 Class Selection Sort.....	5
2.2.4 Output Selection Sort	6
2.2.5 Class Bubble Sort.....	7
2.2.6 Output Bubble Sort	8
2.2.7 Class Insertion Sort (GUI)	8
2.2.8 Output Class Insertion Sort (GUI)	13
BAB III PENUTUP	16
3.1 Kesimpulan.....	16
DAFTAR PUSTKA	17
TUGAS PRAKTIKUM.....	18

BAB I

PENDAHULUAN

1.1 Latar Belakang

Dalam pemrograman, pengurutan data merupakan salah satu proses penting yang sering digunakan untuk mempermudah pencarian dan pengolahan informasi. Data yang tersusun rapi akan membuat proses pengolahan menjadi lebih efisien dan mudah dipahami. Oleh karena itu, diperlukan algoritma sorting untuk mengurutkan data secara sistematis.

Terdapat berbagai metode sorting yang dapat digunakan, seperti Insertion Sort, Selection Sort, dan Bubble Sort. Masing-masing algoritma memiliki cara kerja yang berbeda dalam melakukan pengurutan data. Dengan mempelajari algoritma tersebut, mahasiswa dapat memahami proses pengolahan data secara lebih mendalam.

Selain implementasi dalam bentuk program biasa, praktikum ini juga menerapkan GUI untuk menampilkan proses sorting secara visual. Dengan adanya tampilan langkah demi langkah, pengguna dapat lebih mudah memahami bagaimana proses pengurutan dilakukan oleh algoritma sorting.

1.2 Tujuan Praktikum

Adapun tujuan dari praktikum ini adalah:

1. Memahami konsep dasar algoritma sorting.
2. Mengetahui cara kerja Insertion Sort, Selection Sort, dan Bubble Sort.
3. Mampu mengimplementasikan algoritma sorting dalam Java.
4. Memahami proses pengurutan data langkah demi langkah.
5. Mampu membuat visualisasi sorting menggunakan GUI.

1.3 Manfaat Praktikum

Adapun tujuan dari praktikum ini adalah:

1. Memahami proses pengurutan data dalam pemrograman.
2. Melatih logika algoritma dalam menyusun data.
3. Mengetahui perbedaan cara kerja tiap metode sorting.
4. Mempermudah pemahaman sorting melalui visualisasi GUI.

BAB II

PEMBAHASAN

2.1 Sorting Algorithm

Sorting merupakan proses mengurutkan data berdasarkan urutan tertentu, seperti dari kecil ke besar atau sebaliknya. Dalam pemrograman Java, algoritma sorting digunakan untuk membantu proses pengolahan data agar lebih terstruktur dan mudah diakses.

Pada praktikum ini digunakan tiga metode sorting, yaitu Insertion Sort, Selection Sort, dan Bubble Sort. Ketiga algoritma tersebut memiliki cara kerja yang berbeda, namun tujuan utamanya sama yaitu mengurutkan data.

a. Insertion Sort

Insertion Sort bekerja dengan cara mengambil satu elemen dari array, kemudian membandingkannya dengan elemen sebelumnya hingga menemukan posisi yang tepat. Jika terdapat elemen yang lebih besar, maka elemen tersebut akan digeser ke kanan. Proses ini dilakukan secara berulang sampai seluruh data tersusun dengan benar. Algoritma ini cukup mudah dipahami dan cocok digunakan pada data yang jumlahnya tidak terlalu banyak.

b. Selection Sort

Selection Sort bekerja dengan mencari nilai terkecil dari bagian array yang belum terurut, kemudian menukarnya dengan elemen pada posisi awal. Setelah satu elemen terkecil ditemukan, proses dilanjutkan ke bagian array berikutnya hingga semua data terurut. Algoritma ini memiliki proses yang sederhana karena hanya melakukan pertukaran data setelah menemukan nilai terkecil.

c. Bubble Sort

Bubble Sort bekerja dengan membandingkan dua elemen yang berdekatan. Jika urutannya salah, maka kedua elemen akan ditukar. Proses ini dilakukan berulang kali hingga tidak ada lagi data yang perlu ditukar. Pada setiap putaran, nilai terbesar akan bergerak ke bagian akhir array seperti gelembung (bubble) yang naik ke permukaan.

Selain implementasi dalam bentuk program biasa, praktikum ini juga menggunakan GUI untuk memvisualisasikan proses Insertion Sort secara langkah demi langkah. Dengan adanya tampilan visual, proses perpindahan data menjadi lebih mudah dipahami dibandingkan hanya melihat output teks biasa.

2.2 Kode Pemograman

2.2.1 Class Insertion Sort

```
2 package pekan7_2511531010;
3
4 public class InsertionSort_2511531010 {
5     public static void insertionSort_2511531010(int[] arr_1010) {
6         int n_1010 = arr_1010.length;
7         for (int i_1010 = 1; i_1010 < n_1010; i_1010++) {
8             int key_1010 = arr_1010[i_1010];
9             int j_1010 = i_1010 - 1;
10            while (j_1010 >= 0 && arr_1010[j_1010] > key_1010) {
11                arr_1010[j_1010 + 1] = arr_1010[j_1010];
12                j_1010--;
13            }
14            arr_1010[j_1010 + 1] = key_1010;
15        }
16    }
17
18    public static void main(String[] args) {
19        int arr_1010[] = { 23, 78, 45, 8, 32, 56, 1 };
20        int n_1010 = arr_1010.length;
21        System.out.printf("array yang belum terurut:\n");
22        for (int i_1010 = 0; i_1010 < n_1010; i_1010++)
23            System.out.print(arr_1010[i_1010] + " ");
24        System.out.println("");
25        insertionSort_2511531010(arr_1010);
26        System.out.printf("array yang terurut:\n");
27        for (int i_1010 = 0; i_1010 < n_1010; i_1010++)
28            System.out.print(arr_1010[i_1010] + " ");
```

```

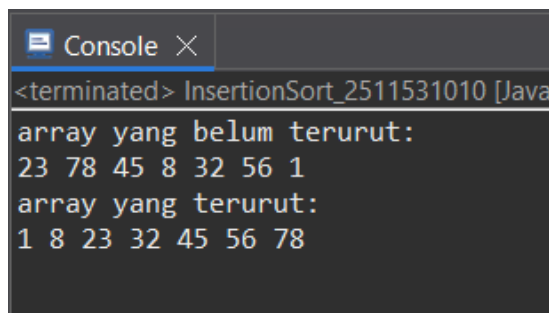
29     System.out.println("");
30 }
31 }

```

Gambar 2.1

Class ini digunakan untuk mengimplementasikan algoritma Insertion Sort. Program melakukan pengurutan data dengan cara mengambil satu elemen, kemudian membandingkannya dengan elemen sebelumnya hingga menemukan posisi yang sesuai. Data yang lebih besar akan digeser ke kanan sampai posisi yang tepat ditemukan.

2.2.2 Output Insertion Sort



```

<terminated> InsertionSort_2511531010 [Java]
array yang belum terurut:
23 78 45 8 32 56 1
array yang terurut:
1 8 23 32 45 56 78

```

Gambar 2.2

Output program pertama menampilkan array sebelum diurutkan, yaitu 23 78 45 8 32 56 1. Setelah proses Insertion Sort dijalankan, array berubah menjadi 1 8 23 32 45 56 78. Perubahan ini menunjukkan bahwa setiap elemen disisipkan ke posisi yang sesuai secara bertahap hingga seluruh data tersusun dari nilai terkecil ke terbesar.

2.2.3 Class Selection Sort

```

2  package pekan7_2511531010;
3
4  public class SelectionSort_2511531010 {
5      public static void selectionSort_2511531010(int[] arr_1010) {
6          int n_1010 = arr_1010.length;
7          for (int i_1010 = 0; i_1010 < n_1010; i_1010++) {
8              int minIndex_1010 = i_1010;

```

```

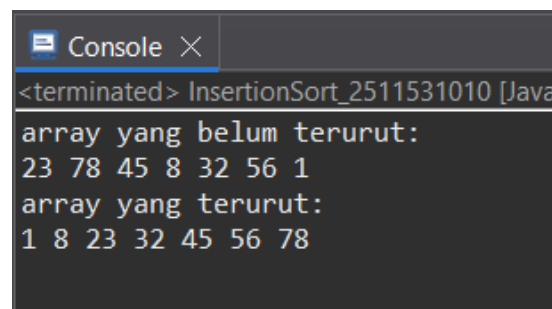
9         for (int j_1010 = i_1010 + 1; j_1010 < n_1010;
j_1010++) {
10             if (arr_1010[j_1010] < arr_1010[minIndex_1010]) {
11                 minIndex_1010 = j_1010;
12             }
13         }
14         int temp_1010 = arr_1010[i_1010];
15         arr_1010[i_1010] = arr_1010[minIndex_1010];
16         arr_1010[minIndex_1010] = temp_1010;
17     }
18 }
19
20 public static void main(String[] args_1010) {
21     int arr_1010[] = { 23, 78, 45, 8, 32, 56, 1 };
22     int n_1010 = arr_1010.length;
23     System.out.printf("array yang belum terurut:\n");
24     for (int i_1010 = 0; i_1010 < n_1010; i_1010++) {
25         System.out.print(arr_1010[i_1010] + " ");
26     }
27     System.out.println("");
28     selectionSort_2511531010(arr_1010);
29     System.out.printf("array yang terurut:\n");
30     for (int i_1010 = 0; i_1010 < n_1010; i_1010++) {
31         System.out.print(arr_1010[i_1010] + " ");
32     }
33     System.out.println("");
34 }
35 }

```

Gambar 2.3

Class ini digunakan untuk mengimplementasikan algoritma Selection Sort. Program bekerja dengan mencari nilai terkecil pada array, kemudian menukarnya dengan elemen pada posisi awal. Proses ini dilakukan berulang hingga seluruh data terurut.

2.2.4 Output Selection Sort



```

<terminated> InsertionSort_2511531010 [Java
array yang belum terurut:
23 78 45 8 32 56 1
array yang terurut:
1 8 23 32 45 56 78

```

Gambar 2.4

Output program menampilkan array awal yang masih acak, kemudian setelah proses Selection Sort selesai, data berubah menjadi urutan yang teratur. Hal ini terjadi karena program terus mencari nilai terkecil dari array yang belum terurut, lalu menukarnya ke posisi depan sampai semua elemen tersusun rapi.

2.2.5 Class Bubble Sort

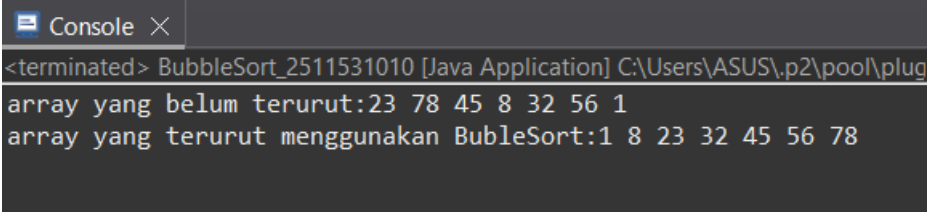
```
2 package pekan7_2511531010;
3
4 public class BubbleSort_2511531010 {
5     public static void bubbleSort_2511531010(int[] arr_1010) {
6         int n_1010 = arr_1010.length;
7         for (int i_1010 = 0; i_1010 < n_1010; i_1010++) {
8             for (int j_1010 = 0; j_1010 < n_1010 - i_1010 - 1;
9                 j_1010++) {
10                 if (arr_1010[j_1010] > arr_1010[j_1010 + 1]) {
11                     int temp_1010 = arr_1010[j_1010];
12                     arr_1010[j_1010] = arr_1010[j_1010 + 1];
13                     arr_1010[j_1010 + 1] = temp_1010;
14                     //
15                     System.out.println("data:"+arr_1010[j_1010]+"
16                     "+arr_1010[j_1010+1]);
17                 }
18             }
19         }
20     }
21
22     public static void main(String[] args_1010) {
23         int arr_1010[] = { 23, 78, 45, 8, 32, 56, 1 };
24         int n_1010 = arr_1010.length;
25         System.out.print("array yang belum terurut:");
26         for (int i_1010 = 0; i_1010 < n_1010; i_1010++) {
27             System.out.print(arr_1010[i_1010] + " ");
28         }
29         System.out.println("");
30         bubbleSort_2511531010(arr_1010);
31         System.out.print("array yang terurut menggunakan
32         BubleSort:");
33         for (int i_1010 = 0; i_1010 < n_1010; i_1010++) {
34             System.out.print(arr_1010[i_1010] + " ");
35         }
36         System.out.println("");
37     }
38 }
```

Gambar 2.5

Class ini digunakan untuk mengimplementasikan Bubble Sort. Program membandingkan dua elemen yang berdekatan, lalu

menukarnya jika posisi tidak sesuai. Proses dilakukan berulang hingga tidak ada lagi data yang perlu ditukar.

2.2.6 Output Bubble Sort

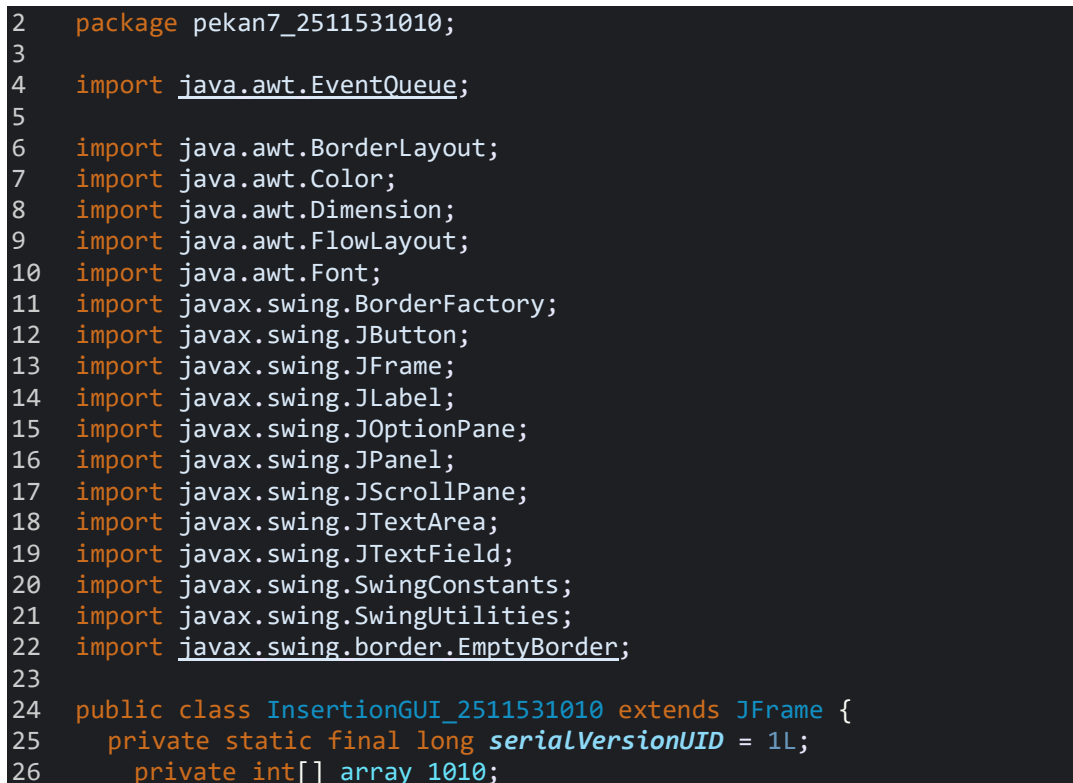


```
<terminated> BubbleSort_2511531010 [Java Application] C:\Users\ASUS\p2\pool\plug
array yang belum terurut:23 78 45 8 32 56 1
array yang terurut menggunakan BubleSort:1 8 23 32 45 56 78
```

Gambar 2.6

Output awal memperlihatkan data sebelum diurutkan. Setelah Bubble Sort dijalankan, data menjadi urut dari kecil ke besar. Hasil ini diperoleh karena setiap elemen dibandingkan dengan elemen di sebelahnya, lalu ditukar jika posisinya salah. Proses ini dilakukan berulang hingga tidak ada lagi pertukaran data.

2.2.7 Class Insertion Sort (GUI)



```
2 package pekan7_2511531010;
3
4 import java.awt.EventQueue;
5
6 import java.awt.BorderLayout;
7 import java.awt.Color;
8 import java.awt.Dimension;
9 import java.awt.FlowLayout;
10 import java.awt.Font;
11 import javax.swing.BorderFactory;
12 import javax.swing.JButton;
13 import javax.swing.JFrame;
14 import javax.swing.JLabel;
15 import javax.swing.JOptionPane;
16 import javax.swing.JPanel;
17 import javax.swing.JScrollPane;
18 import javax.swing.JTextArea;
19 import javax.swing.JTextField;
20 import javax.swing.SwingConstants;
21 import javax.swing.SwingUtilities;
22 import javax.swing.border.EmptyBorder;
23
24 public class InsertionGUI_2511531010 extends JFrame {
25     private static final long serialVersionUID = 1L;
26     private int[] array_1010;
```

```

27     private JLabel[] labelArray_1010;
28     private JButton stepButton_1010, resetButton_1010,
setButton_1010;
29     private JTextField inputField_1010;
30     private JPanel panelArray_1010;
31     private JTextArea stepArea_1010;
32
33     private int i_1010 = 1, j_1010;
34     private boolean sorting_1010 = false;
35     private int stepCount_1010 = 1;
36
37
38     /**
39      * Create the frame.
40      */
41     public InsertionGUI_2511531010() {
42         setTitle("Insertion Sort Langkah per Langkah");
43         setSize(750, 400);
44         setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
45         setLocationRelativeTo(null);
46         setLayout(new BorderLayout());
47
48         // Panel input
49         JPanel inputPanel_1010 = new JPanel(new FlowLayout());
50         inputField_1010 = new JTextField(30);
51         setButton_1010 = new JButton("Set Array");
52         inputPanel_1010.add(new JLabel("Masukkan angka (pisahkan
dengan koma):"));
53         inputPanel_1010.add(inputField_1010);
54         inputPanel_1010.add(setButton_1010);
55
56         // Panel array visual
57         panelArray_1010 = new JPanel();
58         panelArray_1010.setLayout(new FlowLayout());
59
60         // Panel kontrol
61         JPanel controlPanel_1010 = new JPanel();
62         stepButton_1010 = new JButton("Langkah Selanjutnya");
63         resetButton_1010 = new JButton("Reset");
64         stepButton_1010.setEnabled(false);
65         controlPanel_1010.add(stepButton_1010);
66         controlPanel_1010.add(resetButton_1010);
67
68         // Area teks untuk log langkah-langkah
69         stepArea_1010 = new JTextArea(8, 60);
70         stepArea_1010.setEditable(false);
71         stepArea_1010.setFont(new Font("Monospaced", Font.PLAIN,
14));
72         JScrollPane scrollPane_1010 = new
JScrollPane(stepArea_1010);
73
74         // Tambahkan panel ke frame
75         add(inputPanel_1010, BorderLayout.NORTH);
76         add(panelArray_1010, BorderLayout.CENTER);

```

```

77         add(controlPanel_1010, BorderLayout.SOUTH);
78         add(scrollPane_1010, BorderLayout.EAST);
79
80         // Event Set Array
81         setButton_1010.addActionListener(e_1010 ->
            setArrayFromInput());
82
83         // Event Langkah Selanjutnya
84         stepButton_1010.addActionListener(e_1010 -> performStep());
85
86         // Event Reset
87         resetButton_1010.addActionListener(e_1010 -> reset());
88     }
89     private void setArrayFromInput() {
90         String text_1010 = inputField_1010.getText().trim();
91         if (text_1010.isEmpty()) return;
92
93         String[] parts_1010 = text_1010.split(",");
94         array_1010 = new int[parts_1010.length];
95
96         try {
97             for (int k_1010 = 0; k_1010 < parts_1010.length;
            k_1010++) {
98                 array_1010[k_1010] =
                Integer.parseInt(parts_1010[k_1010].trim());
99             }
100         } catch (NumberFormatException e_1010) {
101             JOptionPane.showMessageDialog(this, "Masukkan hanya
                angka yang dipisahkan\n"
102                 + "dengan koma!", "Error",
                JOptionPane.ERROR_MESSAGE);
103             return;
104         }
105         i_1010 = 1;
106         stepCount_1010 = 1;
107         sorting_1010 = true;
108         stepButton_1010.setEnabled(true);
109         stepArea_1010.setText("");
110         panelArray_1010.removeAll();
111
112         labelArray_1010 = new JLabel[array_1010.length];
113         for (int k_1010 = 0; k_1010 < array_1010.length; k_1010++)
114         {
115             labelArray_1010[k_1010] = new
                JLabel(String.valueOf(array_1010[k_1010]));
116             labelArray_1010[k_1010].setFont(new Font("Arial",
                Font.BOLD, 24));
117             labelArray_1010[k_1010].setBorder(BorderFactory.createLineBorder(Color.BLACK));
118             labelArray_1010[k_1010].setPreferredSize(new
                Dimension(50, 50));

```

```

118     labelArray_1010[k_1010].setHorizontalAlignment(SwingConstants.CENTE
R);
119         panelArray_1010.add(labelArray_1010[k_1010]);
120     }
121     panelArray_1010.revalidate();
122     panelArray_1010.repaint();
123 }
124 private void performStep() {
125     if (i_1010 < array_1010.length && sorting_1010) {
126         int key_1010 = array_1010[i_1010];
127         j_1010 = i_1010 - 1;
128
129         StringBuilder stepLog_1010 = new StringBuilder();
130         stepLog_1010.append("Langkah ").append(stepCount_1010)
131             .append(": Memasukkan
132 ").append(key_1010).append("\n");
133         while (j_1010 >= 0 && array_1010[j_1010] > key_1010) {
134             array_1010[j_1010 + 1] = array_1010[j_1010];
135             j_1010--;
136         }
137
138         array_1010[j_1010 + 1] = key_1010;
139
140         updateLabels();
141         stepLog_1010.append("Hasil:
142 ").append(arrayToString(array_1010)).append("\n\n");
143         stepArea_1010.append(stepLog_1010.toString());
144
145         i_1010++;
146         stepCount_1010++;
147
148         if (i_1010 == array_1010.length) {
149             sorting_1010 = false;
150             stepButton_1010.setEnabled(false);
151             JOptionPane.showMessageDialog(this, "Sorting
152 selesail!");
153         }
154     }
155 }
156 private void updateLabels() {
157     for (int k_1010 = 0; k_1010 < array_1010.length; k_1010++)
158     {
159         labelArray_1010[k_1010].setText(String.valueOf(array_1010[k_1010]))
160         ;
161     }
162 }
163 private void reset() {
164     inputField_1010.setText("");
165     panelArray_1010.removeAll();
166     panelArray_1010.revalidate();

```

```

164     panelArray_1010.repaint();
165     stepArea_1010.setText("");
166     stepButton_1010.setEnabled(false);
167     sorting_1010 = false;
168     i_1010 = 1;
169     stepCount_1010 = 1;
170 }
171
172 private String arrayToString(int[] arr_1010) {
173     StringBuilder sb_1010 = new StringBuilder();
174     for (int k_1010 = 0; k_1010 < arr_1010.length; k_1010++) {
175         sb_1010.append(arr_1010[k_1010]);
176         if (k_1010 < arr_1010.length - 1) {
177             sb_1010.append(", ");
178         }
179     }
180     return sb_1010.toString();
181 }
182
183 public static void main(String[] args_1010) {
184     SwingUtilities.invokeLater(() -> {
185         InsertionGUI_2511531010 gui_1010 = new
InsertionGUI_2511531010();
186         gui_1010.setVisible(true);
187     });
188 }
189
190 }

```

Gambar 2.7

Class ini merupakan implementasi GUI (Graphical User Interface) untuk memvisualisasikan proses Insertion Sort secara langkah demi langkah. Program dibuat menggunakan library Swing pada Java, seperti JFrame, JPanel, JButton, JTextField, JLabel, dan JTextArea.

Pada bagian input, pengguna dapat memasukkan angka yang dipisahkan dengan koma melalui text field. Setelah tombol “Set Array” ditekan, data akan diubah menjadi array integer dan ditampilkan secara visual menggunakan label. Program kemudian melakukan proses sorting menggunakan tombol “Langkah Selanjutnya”, sehingga pengguna dapat melihat perubahan urutan array pada setiap langkah sorting.

Method `performStep()` digunakan untuk menjalankan satu langkah proses Insertion Sort, sedangkan method `updateLabels()` digunakan untuk memperbarui tampilan array pada GUI. Selain itu, terdapat tombol Reset untuk menghapus data dan mengembalikan tampilan ke kondisi awal.

GUI ini membantu pengguna memahami proses sorting secara lebih jelas karena setiap perpindahan data dapat dilihat langsung pada layar. Dengan demikian, pengguna tidak hanya melihat hasil akhir sorting, tetapi juga memahami proses yang terjadi pada setiap langkah.

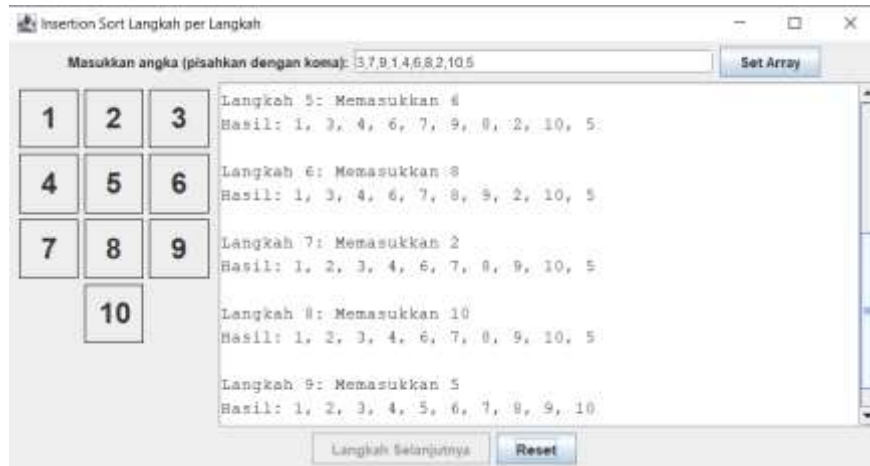
2.2.8 Output Class Insertion Sort (GUI)



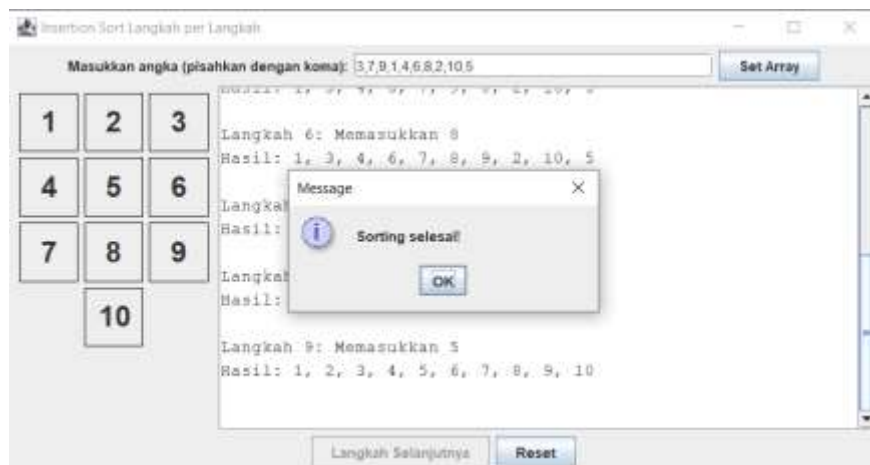
Gambar 2.8

Pada program `InsertionGUI_2511531010`, sebelum tombol “Langkah Selanjutnya” ditekan, pengguna terlebih dahulu memasukkan data angka pada text field, contohnya 3,7,9,1,4,6,8,2,10,5. Setelah tombol “Set Array” ditekan, program akan mengubah input tersebut menjadi array integer dan menampilkannya dalam bentuk kotak-kotak angka pada panel GUI. Pada tahap ini, data masih dalam kondisi belum terurut dan tombol

“Langkah Selanjutnya” mulai dapat digunakan untuk menjalankan proses sorting secara bertahap.



Gambar 2.9



Gambar 2.10

Ketika tombol “Langkah Selanjutnya” ditekan, program akan menjalankan satu langkah proses Insertion Sort. Pada langkah pertama, angka 7 dibandingkan dengan 3, dan karena sudah sesuai urutan maka posisi data tidak berubah. Setelah tombol ditekan lagi, angka 9 juga dibandingkan dengan angka sebelumnya dan tetap berada di posisinya karena sudah lebih besar.

Pada langkah berikutnya, angka 1 akan diproses. Program akan membandingkan angka 1 dengan 9, 7, dan 3, kemudian seluruh angka yang lebih besar akan digeser ke kanan. Hasil array berubah

menjadi 1,3,7,9,4,6,8,2,10,5. Proses ini akan terus berlangsung setiap kali tombol ditekan, sehingga pengguna dapat melihat perubahan posisi angka secara bertahap.

Selanjutnya, angka 4 akan dimasukkan ke posisi yang sesuai di antara angka yang sudah terurut sehingga array berubah menjadi 1,3,4,7,9,6,8,2,10,5. Proses serupa juga terjadi pada angka 6, 8, 2, 10, dan 5. Setiap langkah akan ditampilkan pada area teks beserta hasil array terbaru setelah proses penyisipan dilakukan.

Setelah seluruh langkah selesai, array akan berubah menjadi urutan yang benar yaitu 1,2,3,4,5,6,7,8,9,10. Program kemudian menampilkan pesan bahwa proses sorting telah selesai. Dengan tampilan GUI ini, pengguna dapat memahami bagaimana Insertion Sort bekerja secara bertahap dalam menyusun data dari yang terkecil hingga terbesar.

BAB III

PENUTUP

3.1 Kesimpulan

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa algoritma sorting merupakan metode yang sangat penting dalam pengolahan data karena dapat menyusun data menjadi lebih teratur dan mudah diproses. Dalam praktikum ini digunakan tiga metode sorting, yaitu Insertion Sort, Selection Sort, dan Bubble Sort, yang masing-masing memiliki cara kerja berbeda dalam melakukan pengurutan data.

Melalui implementasi program Java, dapat dipahami bahwa Insertion Sort bekerja dengan menyisipkan data ke posisi yang sesuai, Selection Sort bekerja dengan mencari nilai terkecil untuk ditukar ke posisi awal, sedangkan Bubble Sort bekerja dengan membandingkan dan menukar elemen yang berdekatan secara berulang. Dengan mencoba langsung proses pengurutan tersebut, pemahaman mengenai logika algoritma sorting menjadi lebih jelas.

Selain itu, penggunaan GUI pada Insertion Sort memberikan pengalaman belajar yang lebih interaktif karena proses sorting dapat dilihat secara visual dan bertahap. Pengguna tidak hanya melihat hasil akhir pengurutan, tetapi juga memahami bagaimana perpindahan data terjadi pada setiap langkah. Hal ini membuat konsep sorting menjadi lebih mudah dipahami dibandingkan hanya menggunakan output teks biasa.

DAFTAR PUSTKA

- [1] Oracle Corporation. 2023. *Java Documentation*. Diakses dari <https://docs.oracle.com/javase/8/docs/>
- [2] Oracle Corporation. 2023. *Creating a GUI With Swing*. Diakses dari <https://docs.oracle.com/javase/tutorial/uiswing/>

TUGAS PRAKTIKUM

Soal:

1. Deskripsi Program

Program dikembangkan menggunakan Java GUI (Swing) untuk menampilkan proses pengurutan data mahasiswa secara interaktif.

Program menyediakan beberapa fitur utama, yaitu:

- Input data mahasiswa secara manual melalui form GUI.
- Menyimpan data mahasiswa ke dalam array atau ArrayList object.
- Pemilihan algoritma sorting menggunakan ComboBox.
- Menampilkan visualisasi proses sorting langkah demi langkah.
- Menampilkan hasil akhir pengurutan nama mahasiswa secara ascending (A-Z).

Perbandingan string dilakukan menggunakan method `compareToIgnoreCase()` agar proses sorting tidak membedakan huruf kapital dan huruf kecil.

2. Struktur Program

Program terdiri dari beberapa bagian utama:

A. Kelas ADT Mahasiswa

Class digunakan untuk menyimpan data mahasiswa.

Atribut:

- Nama Mahasiswa (String)
- NIM Mahasiswa (String)
- Program Studi (String)

Method:

- Constructor
- Getter dan Setter

- toString()

B. Kelas GUI Utama

Class GUI digunakan untuk:

- Menampilkan form input data.
- Menampilkan tabel data mahasiswa.
- Menampilkan ComboBox pilihan algoritma sorting.
- Menampilkan proses visualisasi sorting.
- Menjalankan algoritma sorting sesuai pilihan pengguna.

C. Method Sorting

Program menyediakan tiga method sorting terpisah:

- insertionSort()
- selectionSort()
- bubbleSort()

3. Komponen GUI

GUI program terdiri dari beberapa komponen berikut:

- JTextField

Digunakan untuk input nama mahasiswa, NIM, dan program studi.

- JButton

Digunakan untuk:

- Menambahkan data
- Menghapus data
- Memulai sorting

- JComboBox

Digunakan untuk memilih algoritma sorting:

- Insertion Sort
- Selection Sort
- Bubble Sort

- JTable / JTextArea

Digunakan untuk menampilkan:

- Data mahasiswa
- Langkah-langkah proses sorting
- Hasil akhir sorting

4. Alur Program

1. Pengguna menginput data mahasiswa melalui GUI.
2. Data disimpan ke dalam array atau ArrayList.
3. Pengguna memilih algoritma sorting pada ComboBox.
4. Pengguna menekan tombol "Mulai Sorting".
5. Program menjalankan algoritma sorting yang dipilih.
6. Proses sorting ditampilkan visualisasinya langkah demi langkah.
7. Hasil akhir pengurutan ditampilkan pada GUI.

Kode pemograman:

1. Class Mahasiswa_2511531010

```

2. package pekan7_2511531010;
3.
4. public class Mahasiswa_2511531010 {
5.     private String nama_1010;
6.     private String nim_1010;
7.     private String prodi_1010;
8.
9.     public Mahasiswa_2511531010(String nama_1010, String
nim_1010, String prodi_1010) {
10.         this.nama_1010 = nama_1010;
11.         this.nim_1010 = nim_1010;
12.         this.prodi_1010 = prodi_1010;
13.     }
14.
15.     public String getNama_1010() {
16.         return nama_1010;
17.     }
18.

```

```

19.     public String getNim_1010() {
20.         return nim_1010;
21.     }
22.
23.     public String getProdi_1010() {
24.         return prodi_1010;
25.     }
26.
27.     @Override
28.     public String toString() {
29.         return nama_1010;
30.     }
31. }

```

2. Class SortingGUI_2511531010

```

3. package pekan7_2511531010;
4.
5. import javax.swing.*;
6. import javax.swing.table.DefaultTableModel;
7. import java.awt.*;
8. import java.util.ArrayList;
9.
10. public class SortingGUI_2511531010 extends JFrame {
11.     private JTextField txtNama_1010;
12.     private JTextField txtNim_1010;
13.     private JTextField txtProdi_1010;
14.     private JButton btnTambah_1010;
15.     private JButton btnHapus_1010;
16.     private JButton btnSorting_1010;
17.     private JComboBox<String> cbSorting_1010;
18.     private JTable table_1010;
19.     private DefaultTableModel model_1010;
20.     private JTextArea areaProses_1010;
21.     private ArrayList<Mahasiswa_2511531010> list_1010 = new
    ArrayList<>();
22.
23.     public SortingGUI_2511531010() {
24.         setTitle("GUI Sorting Mahasiswa");
25.         setSize(950, 650);
26.         setLocationRelativeTo(null);
27.         setDefaultCloseOperation(EXIT_ON_CLOSE);
28.         setLayout(new BorderLayout());
29.
30.         Color bgColor_1010 = new Color(15, 23, 42);
31.         Color panelColor_1010 = new Color(30, 41, 59);
32.         Color accent_1010 = new Color(0, 191, 255);
33.
34.         JPanel mainPanel_1010 = new JPanel(new BorderLayout());
35.         mainPanel_1010.setBackground(bgColor_1010);
36.
37.         // HEADER
38.         JLabel title_1010 = new JLabel("SORTING DATA MAHASISWA",
    JLabel.CENTER);

```

```

39.         title_1010.setFont(new Font("Poppins", Font.BOLD, 28));
40.         title_1010.setForeground(Color.WHITE);
41.         title_1010.setBorder(BorderFactory.createEmptyBorder(20, 0,
20, 0));
42.
43.         mainPanel_1010.add(title_1010, BorderLayout.NORTH);
44.
45.         // PANEL KIRI
46.         JPanel leftPanel_1010 = new JPanel();
47.         leftPanel_1010.setBackground(panelColor_1010);
48.         leftPanel_1010.setPreferredSize(new Dimension(300, 0));
49.         leftPanel_1010.setLayout(new GridLayout(12, 1, 10, 10));
50.         leftPanel_1010.setBorder(BorderFactory.createEmptyBorder(20,
20, 20, 20));
51.
52.         txtNama_1010 = new JTextField();
53.         txtNim_1010 = new JTextField();
54.         txtProdi_1010 = new JTextField();
55.
56.         cbSorting_1010 = new JComboBox<>();
57.         cbSorting_1010.addItem("Insertion Sort");
58.         cbSorting_1010.addItem("Selection Sort");
59.         cbSorting_1010.addItem("Bubble Sort");
60.
61.         btnTambah_1010 = new JButton("Tambah Data");
62.         btnHapus_1010 = new JButton("Hapus Semua");
63.         btnSorting_1010 = new JButton("Mulai Sorting");
64.
65.         styleButton_1010(btnTambah_1010, accent_1010);
66.         styleButton_1010(btnHapus_1010, Color.RED);
67.         styleButton_1010(btnSorting_1010, new Color(34, 197, 94));
68.
69.         leftPanel_1010.add(label_1010("Nama Mahasiswa"));
70.         leftPanel_1010.add(txtNama_1010);
71.
72.         leftPanel_1010.add(label_1010("NIM"));
73.         leftPanel_1010.add(txtNim_1010);
74.
75.         leftPanel_1010.add(label_1010("Program Studi"));
76.         leftPanel_1010.add(txtProdi_1010);
77.
78.         leftPanel_1010.add(label_1010("Pilih Sorting"));
79.         leftPanel_1010.add(cbSorting_1010);
80.
81.         leftPanel_1010.add(btnTambah_1010);
82.         leftPanel_1010.add(btnHapus_1010);
83.         leftPanel_1010.add(btnSorting_1010);
84.
85.         // PANEL TENGAH
86.         JPanel centerPanel_1010 = new JPanel(new BorderLayout());
87.         centerPanel_1010.setBackground(bgColor_1010);
88.
89.         model_1010 = new DefaultTableModel();
90.         model_1010.addColumn("Nama");

```

```

91.         model_1010.addColumn("NIM");
92.         model_1010.addColumn("Program Studi");
93.
94.         table_1010 = new JTable(model_1010);
95.         table_1010.setRowHeight(25);
96.         table_1010.setFont(new Font("Segoe UI", Font.PLAIN, 14));
97.
98.         JScrollPane scrollTable_1010 = new JScrollPane(table_1010);
99.
100.        areaProses_1010 = new JTextArea();
101.        areaProses_1010.setBackground(panelColor_1010);
102.        areaProses_1010.setForeground(Color.WHITE);
103.        areaProses_1010.setFont(new Font("Consolas", Font.PLAIN,
104.        14));
105.        JScrollPane scrollArea_1010 = new
106.        JScrollPane(areaProses_1010);
107.        scrollArea_1010.setPreferredSize(new Dimension(0, 220));
108.        centerPanel_1010.add(scrollTable_1010,
109.        BorderLayout.CENTER);
110.        centerPanel_1010.add(scrollArea_1010,
111.        BorderLayout.SOUTH);
112.        mainPanel_1010.add(leftPanel_1010, BorderLayout.WEST);
113.        mainPanel_1010.add(centerPanel_1010,
114.        BorderLayout.CENTER);
115.        add(mainPanel_1010);
116.
117.        btnTambah_1010.addActionListener(e_1010 ->
118.        tambahData_1010());
119.        btnHapus_1010.addActionListener(e_1010 ->
120.        hapusData_1010());
121.        btnSorting_1010.addActionListener(e_1010 ->
122.        prosesSorting_1010());
123.    }
124.
125.    private JLabel label_1010(String text_1010) {
126.        JLabel lbl_1010 = new JLabel(text_1010);
127.        lbl_1010.setForeground(Color.WHITE);
128.        lbl_1010.setFont(new Font("Segoe UI", Font.BOLD, 14));
129.        return lbl_1010;
130.    }
131.
132.    private void styleButton_1010(JButton btn_1010, Color
133.    color_1010) {
134.        btn_1010.setBackground(color_1010);
135.        btn_1010.setForeground(Color.WHITE);
136.        btn_1010.setFocusPainted(false);
137.        btn_1010.setFont(new Font("Segoe UI", Font.BOLD, 14));
138.    }
139.
140.    private void tambahData_1010() {
141.        String nama_1010 = txtNama_1010.getText();
142.        String nim_1010 = txtNim_1010.getText();
143.        String prodi_1010 = txtProdi_1010.getText();

```

```

136.
137.         Mahasiswa_2511531010 mhs_1010 =
138.             new Mahasiswa_2511531010(nama_1010, nim_1010,
prodi_1010);
139.         list_1010.add(mhs_1010);
140.         model_1010.addRow(new Object[]{
141.             nama_1010, nim_1010, prodi_1010
142.         });
143.
144.         txtNama_1010.setText("");
145.         txtNim_1010.setText("");
146.         txtProdi_1010.setText("");
147.     }
148.
149.     private void hapusData_1010() {
150.         list_1010.clear();
151.         model_1010.setRowCount(0);
152.         areaProses_1010.setText("");
153.     }
154.     private void prosesSorting_1010() {
155.         areaProses_1010.setText("");
156.         String pilihan_1010 =
157.             cbSorting_1010.getSelectedItem().toString();
158.         ArrayList<Mahasiswa_2511531010> temp_1010 =
159.             new ArrayList<>(list_1010);
160.         if (pilihan_1010.equals("Insertion Sort")) {
161.             insertionSort(temp_1010);
162.         }
163.         else if (pilihan_1010.equals("Selection Sort")) {
164.             selectionSort(temp_1010);
165.         }
166.         else {
167.             bubbleSort(temp_1010);
168.         }
169.
170.         model_1010.setRowCount(0);
171.
172.         for (Mahasiswa_2511531010 m_1010 : temp_1010) {
173.
174.             model_1010.addRow(new Object[]{
175.                 m_1010.getNama_1010(),
176.                 m_1010.getNim_1010(),
177.                 m_1010.getProdi_1010()
178.             });
179.         }
180.     }
181.
182.     // INSERTION SORT
183.     private void insertionSort(ArrayList<Mahasiswa_2511531010>
data_1010) {
184.         areaProses_1010.append("=== INSERTION SORT ===\n\n");
185.         for (int i_1010 = 1; i_1010 < data_1010.size(); i_1010++)
{

```

```

186.         Mahasiswa_2511531010 key_1010 =
data_1010.get(i_1010);
187.         int j_1010 = i_1010 - 1;
188.         while (j_1010 >= 0 &&
189.             data_1010.get(j_1010).getNama_1010()
190.             .compareToIgnoreCase(key_1010.getNama_1010())
> 0) {
191.             data_1010.set(j_1010 + 1, data_1010.get(j_1010));
192.             j_1010--;
193.         }
194.
195.         data_1010.set(j_1010 + 1, key_1010);
196.
197.         tampilLangkah(data_1010, i_1010);
198.     }
199. }
200.
201. // SELECTION SORT
202. private void selectionSort(ArrayList<Mahasiswa_2511531010>
data_1010) {
203.     areaProses_1010.append("=== SELECTION SORT ===\n\n");
204.     for (int i_1010 = 0; i_1010 < data_1010.size() - 1;
i_1010++) {
205.         int min_1010 = i_1010;
206.         for (int j_1010 = i_1010 + 1; j_1010 <
data_1010.size(); j_1010++) {
207.             if (data_1010.get(j_1010).getNama_1010()
208.                 .compareToIgnoreCase(
209. data_1010.get(min_1010).getNama_1010()) < 0) {
210.                 min_1010 = j_1010;
211.             }
212.         }
213.
214.         Mahasiswa_2511531010 temp_1010 =
data_1010.get(i_1010);
215.         data_1010.set(i_1010, data_1010.get(min_1010));
216.         data_1010.set(min_1010, temp_1010);
217.
218.         tampilLangkah(data_1010, i_1010 + 1);
219.     }
220. }
221.
222. // BUBBLE SORT
223. private void bubbleSort(ArrayList<Mahasiswa_2511531010>
data_1010) {
224.     areaProses_1010.append("=== BUBBLE SORT ===\n\n");
225.     for (int i_1010 = 0; i_1010 < data_1010.size() - 1;
i_1010++) {
226.         for (int j_1010 = 0; j_1010 < data_1010.size() -
i_1010 - 1; j_1010++) {
227.             if (data_1010.get(j_1010).getNama_1010()
228.                 .compareToIgnoreCase(

```

```

229.                                     data_1010.get(j_1010 +
230. 1).getNama_1010()) > 0) {
231.                                     Mahasiswa_2511531010 temp_1010 =
232. data_1010.get(j_1010);
233.                                     data_1010.set(j_1010, data_1010.get(j_1010 +
234. 1));
235.                                     data_1010.set(j_1010 + 1, temp_1010);
236.                                     }
237.                                     }
238.                                     }
239.                                     tampilLangkah(data_1010, i_1010 + 1);
240.                                     }
241.                                     }
242.                                     }
243.                                     }
244.                                     }
245.                                     }
246.                                     }
247.                                     }
248.                                     }
249.                                     }
250.                                     }
251.                                     }
252.                                     }
253.                                     }
254.                                     }
255.                                     }

```

3. Output

Nama	NIM	Program Studi
Farel	2511531010	Informatika
Irfan	2511531008	Informatika
Aufa	2511531012	Informatika
Nanda	2511532008	Informatika

Insertion Sort

GUI Sorting Mahasiswa

SORTING DATA MAHASISWA

Nama	NIM	Program Studi
Aufa	2511531012	Informatika
Fharel	2511531010	Informatika
Irfan	2511531008	Informatika
Nanda	2511532008	Informatika

Nama Mahasiswa

NIM

Program Studi

Pilih Sorting

Insertion Sort

Tambah Data

Hapus Semua

Mulai Sorting

=== INSERTION SORT ===

Langkah 1 : Fharel | Irfan | Aufa | Nanda |

Langkah 2 : Aufa | Fharel | Irfan | Nanda |

Langkah 3 : Aufa | Fharel | Irfan | Nanda |

Selection Sort

GUI Sorting Mahasiswa

SORTING DATA MAHASISWA

Nama	NIM	Program Studi
Aufa	2511531012	Informatika
Fharel	2511531010	Informatika
Irfan	2511531008	Informatika
Nanda	2511532008	Informatika

Nama Mahasiswa

NIM

Program Studi

Pilih Sorting

Selection Sort

Tambah Data

Hapus Semua

Mulai Sorting

=== SELECTION SORT ===

Langkah 1 : Aufa | Irfan | Fharel | Nanda |

Langkah 2 : Aufa | Fharel | Irfan | Nanda |

Langkah 3 : Aufa | Fharel | Irfan | Nanda |

Bubble Sort

GUI Sorting Mahasiswa

SORTING DATA MAHASISWA

Nama	NIM	Program Studi
Aufa	2511531012	Informatika
Fharel	2511531010	Informatika
Irfan	2511531008	Informatika
Nanda	2511532008	Informatika

Nama Mahasiswa

NIM

Program Studi

Pilih Sorting

Bubble Sort

Tambah Data

Hapus Semua

Mulai Sorting

=== BUBBLE SORT ===

Langkah 1 : Fharel | Aufa | Irfan | Nanda |
Langkah 2 : Aufa | Fharel | Irfan | Nanda |
Langkah 3 : Aufa | Fharel | Irfan | Nanda |